Guía del sistema · Qué hace, qué usa y cómo funciona

Dashboard de telemetría del robot Unitree Go2 · TP05 Desarrollo de Aplicaciones II · UADE

1. Qué hace el sistema

Es una página web que muestra **en tiempo real** el estado de un robot cuadrúpedo Unitree Go2 y permite guardar muestras y exportarlas a un archivo CSV. Funciona como un tablero de control (dashboard) de solo lectura: observa el robot pero nunca le manda órdenes.

Panel	Qué muestra y para qué sirve
Motores	12 motores (3 por pata: cadera, muslo y pantorrilla). Temperatura con semáforo (verde menos de 40 °C, amarillo de 40 a 60, rojo más de 60), ángulo, velocidad y torque. Sirve para detectar un motor que se sobrecalienta antes de que se dañe.
IMU	La unidad de medición inercial dice cómo está inclinado el robot: *roll* (giro lateral), *pitch* (cabeceo) y *yaw* (rumbo), más la aceleración. Se ve con un horizonte artificial, una brújula y un gráfico de las últimas 300 muestras.
Batería (BMS)	El sistema de gestión de batería informa la carga (%), la corriente (mA), la temperatura y el voltaje de cada celda. Una diferencia grande entre celdas indica que están desbalanceadas.
Fuerzas por pata	Indica qué patas están apoyadas en el suelo y cuáles en el aire (FR, FL, RR, RL). Permite seguir la marcha.
Gemelo digital	Dibujo del robot cuyas patas se mueven con los ángulos reales de los motores y cuyas articulaciones se pintan según su temperatura: se ve de un vistazo cómo camina y dónde está el problema.
Conexión	Fuente de datos en uso, mensajes por segundo, tráfico y cuánto hace que llegó el último dato.
Captura y CSV	«Capturar muestra» guarda una foto de todos los valores; «Exportar CSV» descarga todas las muestras en un archivo que se abre en Excel.

Si se corta la conexión, aparece un aviso rojo, los paneles quedan atenuados con los últimos valores y el sistema reintenta solo. Si algún motor supera 60 °C, avisa y guarda una muestra automáticamente.

2. Qué tecnologías usa

Tecnología	Para qué se usa
HTML	Estructura de la página: paneles, tablas y botones.
CSS	Aspecto: colores, grilla, tema oscuro y claro, adaptación a celular.
JavaScript (sin framework)	Toda la lógica: conectarse, recibir datos, actualizar paneles, armar el CSV. Se usa «JavaScript puro» para que no haya nada que instalar.
Chart.js	Librería de gráficos (se carga desde internet por CDN) para las curvas de IMU, temperatura y celdas.
FastAPI (Python)	Es el *servidor* que provee la cátedra: lee el robot o su simulador y publica los datos. No es parte de este trabajo, pero el dashboard depende de él.
Node.js	Solo para ejecutar las 34 pruebas automáticas; no se necesita para usar el dashboard.
Git y GitHub	Control de versiones: historial de cambios y publicación del código.

3. Los conceptos que aparecen

Cliente-servidor

Es un modelo donde dos programas se reparten el trabajo. El **servidor** tiene los datos (aquí, el estado del robot) y espera pedidos; el **cliente** los solicita y los presenta. En este sistema el servidor es el backend FastAPI y el cliente es el dashboard que corre en el navegador. Son independientes: se podría reemplazar uno sin tocar el otro, mientras se respete el contrato (la API).

API, HTTP y JSON

La **API** es la lista de «direcciones» que el servidor ofrece, por ejemplo `/telemetria`. **HTTP** es el protocolo con el que se piden (el mismo de cualquier página web) y **JSON** es el formato de texto en que viene la respuesta, con la forma `{"motores": [...], "imu": {...}, "bms": {...}, "fuerzas": {...}}`. El dashboard valida cada respuesta contra esa forma y descarta lo que no la cumple.

Polling (sondeo)

El cliente **pregunta** una y otra vez: «¿qué hay de nuevo?». Aquí, un `GET /telemetria` cada 250 ms. Es como refrescar una página muy rápido. Ventajas: es simple y funciona en cualquier red. Desventajas: cada dato exige un pedido completo, y siempre puede llegar un poco atrasado (hasta un intervalo).

WebSocket (push)

El cliente abre **una conexión que queda abierta** y el servidor **empuja** los datos apenas los tiene, unos 10 por segundo. Es como una llamada telefónica en lugar de mandar cartas. Ventajas: datos más frescos y sin repetir encabezados. Desventajas: hay que manejar la reconexión si se corta, y esta versión del servidor no admite parámetros.

Modo automático

El dashboard usa WebSocket y, si falla, cae a polling mientras reintenta el WebSocket cada vez con más espera (de 1 a 10 s). Cuando se recupera, vuelve solo. Así el usuario ve datos aunque el WebSocket no esté disponible.

Patrón Observer

Un **patrón de diseño** es una solución probada a un problema típico. En Observer hay un **publicador** (*Subject*) que avisa a sus **suscriptores** (*Observers*) cuando algo cambia. Aquí el publicador es la clase `Telemetria`, que anuncia «llegó un dato» o «cambió la conexión», y cada panel (motores, IMU, batería…) se suscribe y se actualiza solo. Ningún panel sabe de dónde vienen los datos, por eso WebSocket y polling se pueden intercambiar sin tocarlos.

Buffer circular

Una lista de tamaño fijo (300 muestras) en la que, al llenarse, entra la nueva y sale la más antigua. Evita que la memoria crezca sin límite al dejar el dashboard horas abierto y mantiene el gráfico fluido.

Histéresis

Para que una alerta no se dispare y apague sin parar cuando un motor oscila alrededor de 60 °C, se activa al superar 60 °C pero se rearma recién al bajar de 58 °C.

Blob y exportación sin servidor

Un **Blob** es un archivo en memoria del navegador. El CSV se arma con los datos ya capturados y se descarga directamente, sin enviar nada al servidor.

CORS

Regla de seguridad del navegador que limita a qué servidores puede pedir datos una página. El servidor de la cátedra la tiene abierta para facilitar el trabajo en el laboratorio; por eso debe usarse solo en una red cerrada.

4. Cómo se conecta todo (resumen del recorrido de un dato)

1. El robot (o su simulador) genera el estado de motores, IMU, batería y patas.
2. El servidor FastAPI lo publica por `GET /telemetria` y por `WS /ws`.
3. La clase `Telemetria` del dashboard lo recibe (por push o por polling), lo valida y lo publica como evento `dato`.
4. Cada panel suscripto actualiza su parte de la pantalla; los gráficos agregan el punto al buffer circular.
5. Si el usuario pulsa «Capturar muestra», el estado actual se guarda en una lista; «Exportar CSV» convierte la lista en un archivo y lo descarga.

5. Cómo se usa

- Levantar el servidor: `python arrancar_api.py --demo --robot go2` (necesita Python 3.11 o superior con `fastapi uvicorn websockets`).
- Abrir `dashboard.html` en el navegador (se detecta el servidor en `localhost:8001`), o escribir otra dirección en el campo «Servidor».
- Para ver el semáforo en rojo con el simulador: agregar `?forzarTemp=25` a la dirección de la página.